Operating System Fundamentals · Chapter 05

CPU Scheduling

How the OS decides which ready process gets the CPU next, how we judge that decision, and four classic algorithms worked step by step.

Chapter 5 of the notebook

BIG QUESTION

Five programs want the CPU right now. Who goes first?

Today's route

What we'll cover today

1. **Basic concepts**
   Bursts, scheduler, preemption, dispatcher
2. **Scheduling criteria**
   How we judge a schedule
3. **FCFS and SJF**
   First-come first-served, shortest-job first
4. **Priority and Round Robin**
   Importance, and taking turns

Part 01

Basic concepts

THINK FIRST

At the supermarket, should the person with one item wait behind a full trolley?

discuss for one minute before the next slide

CPU–I/O burst cycle

Programs alternate: compute, wait, compute



Why it matters

While one process waits for I/O, the CPU can run another. That's how multitasking gets maximum CPU utilisation.

↺ Recall • I/O-bound vs CPU-bound, Ch 4

I/O-bound: many short CPU bursts. CPU-bound: few very long CPU bursts.

The CPU scheduler

When does it have to decide?

↺ Recall · short-term scheduler, Ch 4

It selects one of the ready processes in memory and gives it the CPU.

1	2	3	4
Running → waiting	**Running → ready**	**Waiting → ready**	**Terminates**
e.g. asks for I/O	e.g. timer interrupt	e.g. I/O finished	process is done

only 1 and 4? → non-preemptive. Also 2 and 3? → preemptive.

Two styles

Preemptive vs non-preemptive

Preemptive

- The OS can take the CPU away before the process finishes its burst
- Example: modern OSes (UNIX, Linux, Windows)

Non-preemptive

- The process keeps the CPU until it goes running → waiting, or running → terminated
- Example: MS Windows 3.1

preemptive = the teacher can interrupt you; non-preemptive = you talk until you're done

The dispatcher

The scheduler chooses; the dispatcher delivers

1. **Switch context**
   save / load PCBs
2. **Switch to user mode**
   from monitor mode
3. **Jump**
   to the right place in the user program

↺ **Recall · context switch, Ch 4 · dual mode, Ch 2**

Both were explained there; the dispatcher simply performs them.

Dispatch latency

The time the dispatcher takes to stop one process and start another. Keep it small!

Pop quiz • click to reveal

A scheduler only makes decisions when a process blocks for I/O or terminates. It is…

A Preemptive

B Non-preemptive

C Round Robin

D A dispatcher

Pop quiz • click to reveal

A scheduler only makes decisions when a process blocks for I/O or terminates. It is…

- **A** Preemptive
- **B** Non-preemptive
- **C** Round Robin
- **D** A dispatcher

Answer B. Deciding only at running → waiting and termination is the definition of non-preemptive scheduling.

Part 02

Scheduling criteria

THINK FIRST

What makes a scheduler "good"? A busy CPU, or happy users? Can you have both?

discuss for one minute before the next slide

Five yardsticks

How we judge a schedule

MAXIMISE ↑

CPU utilisation

Keep the CPU as busy as possible.

MAXIMISE ↑

Throughput

Processes completed per time unit.

MINIMISE ↓

Turnaround time

Time to execute a particular process, from submission to completion.

MINIMISE ↓

Waiting time

Time spent waiting in the ready queue.

MINIMISE ↓

Response time

From request to first response (not full output). Key for time-sharing.

FAIRNESS

Also consider

Minimise the maximum response time, and the variance of response times.

Your toolkit

Two formulas for every example



Turnaround time

= completion time − arrival time



Waiting time

= turnaround time − burst time



Average waiting time = sum of all waiting times ÷ number of processes

we'll use these on every Gantt chart that follows

Part 03

First-Come First-Served & Shortest-Job First

THINK FIRST

Same three jobs, two different orders. Can the order alone change how long everyone waits?

discuss for one minute before the next slide

Algorithm 1

First-Come, First-Served (FCFS)



How it works

- Processes wait in one line, in arrival order
- New arrivals join the back of the line
- The process at the front gets the CPU



Verdict

- Easy to understand and build
- But the average waiting time is often long!

just like the queue at the bank: no skipping

FCFS • example 1

Long job first: everyone waits

Process	Burst time
P1	24
P2	3
P3	3

Waiting times

- P1 = 0
- P2 = 24
- P3 = 27



Average waiting time = (0 + 24 + 27) / 3 = 17

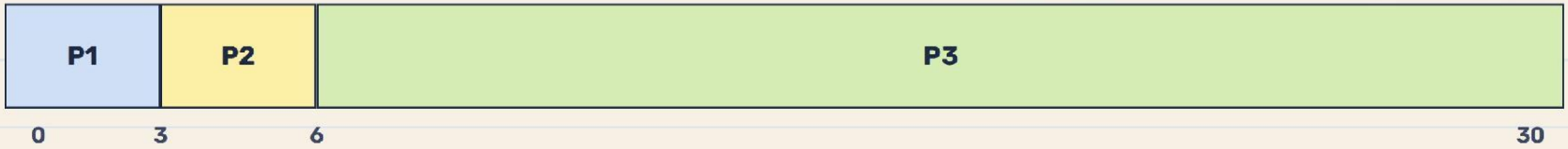
FCFS • example 2

Short jobs first: much better

Process	Burst time
P1	3
P2	3
P3	24

Waiting times

- P1 = 0
- P2 = 3
- P3 = 6



Average waiting time = (0 + 3 + 6) / 3 = 3

Same jobs, different order: 17 vs 3.

The convoy effect: short processes stuck behind one long process, like cars behind a slow truck.

Algorithm 2

Shortest-Job First (SJF)

The rule

Look at the length of each process's next CPU burst. Run the shortest first.

Non-preemptive

Once a process has the CPU, it keeps it until its burst completes.

Preemptive (SRTF)

A new arrival shorter than the running process's remaining time preempts it: Shortest-Remaining-Time-First.

SJF is optimal: it gives the minimum average waiting time for a given set of processes.

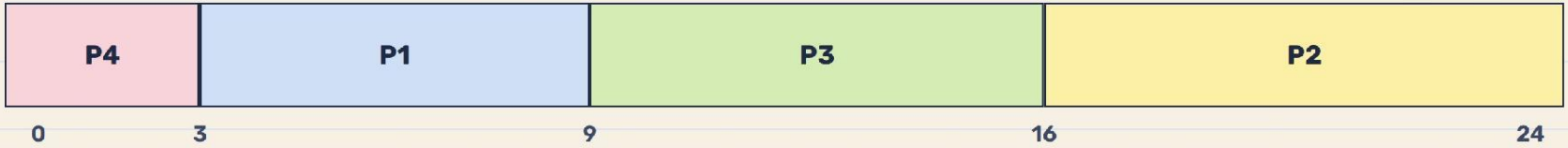
SJF · all arrive at 0

Shortest first: P4, P1, P3, P2

Process	Burst time
P1	6
P2	8
P3	7
P4	3

Waiting times

- P1 = 3
- P2 = 16
- P3 = 9
- P4 = 0



Average waiting time = (3 + 16 + 9 + 0) / 4 = 7

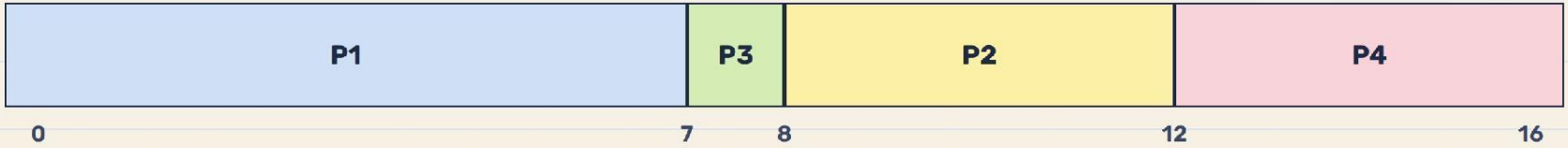
SJF • non-preemptive, with arrivals

Arrivals change the picture

Process	Arrival	Burst
P1	0.0	7
P2	2.0	4
P3	4.0	1
P4	5.0	4

Waiting times

- P1 = 0
- P2 = 8 − 2 = 6
- P3 = 7 − 4 = 3
- P4 = 12 − 5 = 7



Average waiting time = (0 + 6 + 3 + 7) / 4 = 4

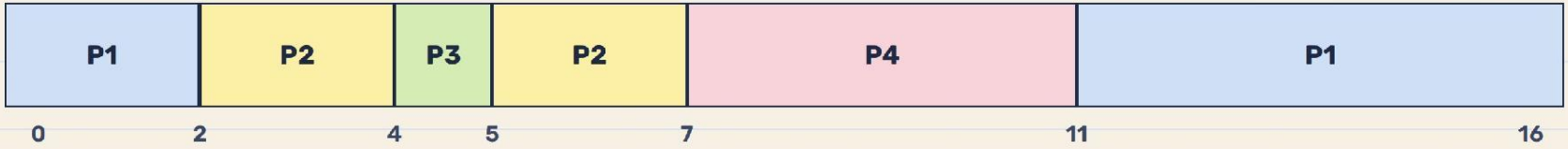
SJF · preemptive (SRTF)

Preemption brings it down to 3

Process	Arrival	Burst
P1	0.0	7
P2	2.0	4
P3	4.0	1
P4	5.0	4

Waiting times

- P1 = 16 − 7 − 0 = 9
- P2 = 7 − 4 − 2 = 1
- P3 = 0
- P4 = 11 − 4 − 5 = 2



Average waiting time = (9 + 1 + 0 + 2) / 4 = 3

Pop quiz • click to reveal

Which algorithm gives the minimum average waiting time for a given set of processes?

A FCFS

B SJF

C Round Robin with a tiny quantum

D Random

Pop quiz • click to reveal

Which algorithm gives the minimum average waiting time for a given set of processes?

A FCFS

B SJF

C Round Robin with a tiny quantum

D Random

Answer B. SJF is provably optimal for average waiting time. Its problem is knowing the burst lengths in advance.

Part 04

Priority & Round Robin

THINK FIRST

In an emergency room, who's seen first? And in a classroom Q&A, how do you make sure everyone gets a turn?

discuss for one minute before the next slide

Algorithm 3

Priority scheduling

The rule

Each process has a priority number. The CPU goes to the highest priority. Here: smallest integer = highest priority.

Two flavours

- Preemptive
- Non-preemptive

SJF in disguise

SJF is priority scheduling where the priority is the predicted next CPU burst.

Priority · example

Smallest number runs first

Process	Burst	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Waiting times

- P1 = 6
- P2 = 0
- P3 = 16
- P4 = 18
- P5 = 1



Average waiting time = 41 / 5 = 8.2

The danger

Starvation, and the cure: aging



Starvation

Low-priority processes may never execute, because something more important always arrives first.



Aging

As time passes, gradually increase the priority of processes that have been waiting. Eventually everyone runs.

Algorithm 4

Round Robin (RR): everyone takes turns

The rule

Each process gets a small time quantum q (usually 10–100 ms). When it expires, the process is preempted and goes to the back of the ready queue.

Fair by design

With n processes, each gets 1/n of the CPU in chunks of at most q. No process waits more than (n − 1) × q.

q very large

RR becomes FCFS

q very small

Context switches eat the CPU. q must be large compared with the switch time.

RR • quantum = 20

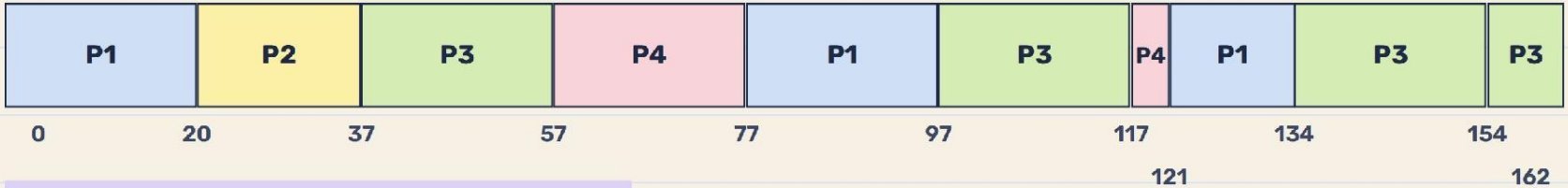
Turn by turn, 20 at a time

Process	Burst time
P1	53
P2	17
P3	68
P4	24

Waiting times

- P1 = 134 − 53 = 81
- P2 = 37 − 17 = 20
- P3 = 162 − 68 = 94
- P4 = 121 − 24 = 97

higher average turnaround than SJF, but much better response



Average waiting time = 292 / 4 = 73

Pop quiz • click to reveal

Round Robin with a very large time quantum behaves like…

A SJF

B FCFS

C Priority scheduling

D Nothing: it stops working

Pop quiz • click to reveal

Round Robin with a very large time quantum behaves like…

A SJF

B FCFS

C Priority scheduling

D Nothing: it stops working

Answer B. If q is bigger than every burst, no one is ever preempted: first come, first served.

Side by side

Four algorithms in one table

Algorithm	Preemptive?	Strength	Weakness
FCFS	No	Simple, fair order	Convoy effect, long waits
SJF / SRTF	Either	Minimum average waiting time	Needs burst prediction; long jobs can starve
Priority	Either	Important work first	Starvation (fix: aging)
Round Robin	Yes	Good response time, fair	Higher turnaround; q must be chosen well

Wrap-up

What to remember

OS JOURNAL · CHAPTER 5

- ✓ Processes alternate CPU bursts and I/O waits.
- ✓ The dispatcher performs the context switch the scheduler chose.
- ✓ SJF is optimal on average waiting; priority needs aging.
- ✓ Non-preemptive decides only on block or exit; preemptive can interrupt.
- ✓ Waiting = turnaround − burst; turnaround = completion − arrival.
- ✓ RR trades turnaround for response; q decides everything.

Next up *Chapter 6 · Deadlocks: when processes wait for each other forever*